

**CESS Program
Fall 2024**



Introduction to Machine Learning

CSE 381

Heart Disease Prediction

Submitted To:

Dr. Mahmoud Khalil

Eng. Engy Ahmed

Team Members:

Siveen Said Sayed Ahmed

21P0149

Mariam Mahmoud Abdou

21P0214

Malak Mohamed Salem

21P0277

Introduction

This project presents a comprehensive study on predicting heart disease using machine learning techniques. It aims to analyze a dataset containing various medical and physiological parameters to predict the presence of heart disease. The project encompasses data preprocessing, exploratory data analysis, feature engineering, model selection, evaluation, and reasoning behind every step.

Dataset Description

Features

The dataset contains information on various attributes representing patient data:

- **Age**
- **Sex:** "M" for male, "F" for female.
- **ChestPainType:** Describing the type of chest pain experienced. ("TA" for typical angina, "ATA" for atypical angina, "NAP" for non-anginal pain, "ASY" for asymptomatic). ^[1]
- **RestingBP:** Representing the resting blood pressure (in mmHg).
- **Cholesterol:** Representing the serum cholesterol level (in mg/dL).
- **FastingBS:** Indicating if fasting blood sugar is greater than 120 mg/Dl (1 = Yes, 0 = No).
- **RestingECG:** Describing the results of a resting electrocardiogram (ECG) test, which measures the electrical activity of the heart. ("Normal," "ST," "LVH" for left ventricular hypertrophy). ^[2]
- **MaxHR:** Representing the maximum heart rate achieved.
- **ExerciseAngina:** Indicating if exercise-induced angina is present. (1 = Yes, 0 = No).
- **Oldpeak:** Representing the ST depression induced by exercise relative to rest (often used to measure ischemia).
- **ST_Slope:** Representing the slope of the peak exercise ST segment. ("Up," "Flat," "Down").
- **HeartDisease:** Indicating the presence of heart disease. (1 = Yes, 0 = No).

Features Nature

Categorical features: Sex, ChestPainType, RestingECG, ST_Slope.

Numerical features: Age, RestingBP, Cholesterol, FastingBS, MaxHR, ExerciseAngina, Oldpeak.

Target: HeartDisease.

Data Preprocessing

Handling Range of Values

For each feature in the dataset, its range was examined to identify potential anomalies. If a value was found to be outside the expected range, it was either considered a missing value or corrected based on logical inference. This ensured the data quality was upheld for further analysis. A detailed breakdown of feature ranges and identified issues has been documented separately to maintain transparency and guide preprocessing steps.

Range of values

- **Age Range:** 0 to 120 years.
- **Sex Values:** ['M', 'F'].
- **ChestPainType Values:** ['ATA', 'NAP', 'ASY', 'TA'].
- **RestingBP (Resting Blood Pressure) Range:** 0 to 250 mmHg.
Note: Values below 90 or above 200 are clinically abnormal.
- **Cholesterol:** Range: 0 to 600 mg/dL.
Note: Values above 300 are rare and usually indicate a serious condition.
- **FastingBS (Fasting Blood Sugar) Values:** [0, 1].
- **RestingECG Values:** ['Normal', 'ST', 'LVH'].
- **MaxHR Range:** 0 to 220 bpm.
Note: Typical values range from 60 bpm (resting) to around 200 bpm (during exercise).
- **ExerciseAngina Values:** [0, 1].
- **Oldpeak Range:** 0 to 6.5 mm.
Note: Values above 5.0 are rare and clinically significant.
- **ST_Slope Values:** ['Up', 'Flat', 'Down']

Issues in data

- Max observed value in **Cholesterol** is 603. Since only one value is outside the expected range. Only one value is outside the expected range, this value is valid but indicates high risk, it is retained in the dataset.
- Negative values observed in **Oldpeak**. Since it cannot be negative, the negative values are considered entry errors and are converted to positive values that are inside the absolute range.

Missing Values

The dataset initially had no missing values; however, outliers were identified mostly to be replaced with null values. These null values were subsequently handled using the KNN-imputer.

Handling Outliers

Outlier detection was performed for both categorical and numerical data to ensure data quality and reliability.

Numerical data

The Z-score method was applied to detect outliers. This method identifies values that deviate significantly from the mean, typically using a threshold of $|Z| > 3$.

These outliers were added to a list for further handling, such as replacement, removal, or adjustment based on their impact on the analysis and model performance.

```
def detect_outliers(df, feature, threshold=3):
    z_scores = zscore(df[feature])
    outliers = (z_scores > abs(threshold))
    print(f"{outliers.sum()} outliers detected in '{feature}':")
    print(df[feature][outliers])
    print("-" * 50)
    return df[feature][outliers]
```

The maximum number of outliers detected in a column was 7 out of 918 rows. Since this is a small proportion, and they are meaningful and provide important insights (as rare but valid occurrences in medical datasets), they will be retained in the dataset.

Categorical data

Infrequent categories were identified in each feature based on a threshold of 5% of the dataset. Categories present less frequently than the threshold were considered potential outliers and added to a list for further analysis.

```
def detect_infrequent_categories(df, feature, threshold = 0.05*len(df)):
    value_counts = df[feature].value_counts()
    infrequent = value_counts[value_counts < threshold].index
    return infrequent
```

```
categorical_outliers = {}  
for feature in df.select_dtypes(include=['object']):  
    olist = detect_infrequent_categories(df, feature)  
    categorical_outliers[feature] = olist  
categorical_outliers
```

Handling these categories typically involves decisions such as grouping them into an "Other" class, retaining them if they represent valid rare occurrences, or treating them as noise based on their significance. However, no outliers were detected, as all values were frequent and met the defined threshold criteria.

Encoding Categorical Variables

Since machine learning algorithms generally require numerical input, categorical variables needed to be converted into a numeric format through encoding. This transformation ensures the data is suitable for the algorithms.

For this purpose, **Label Encoding** was applied, which assigns a unique numerical value to each category within a variable.

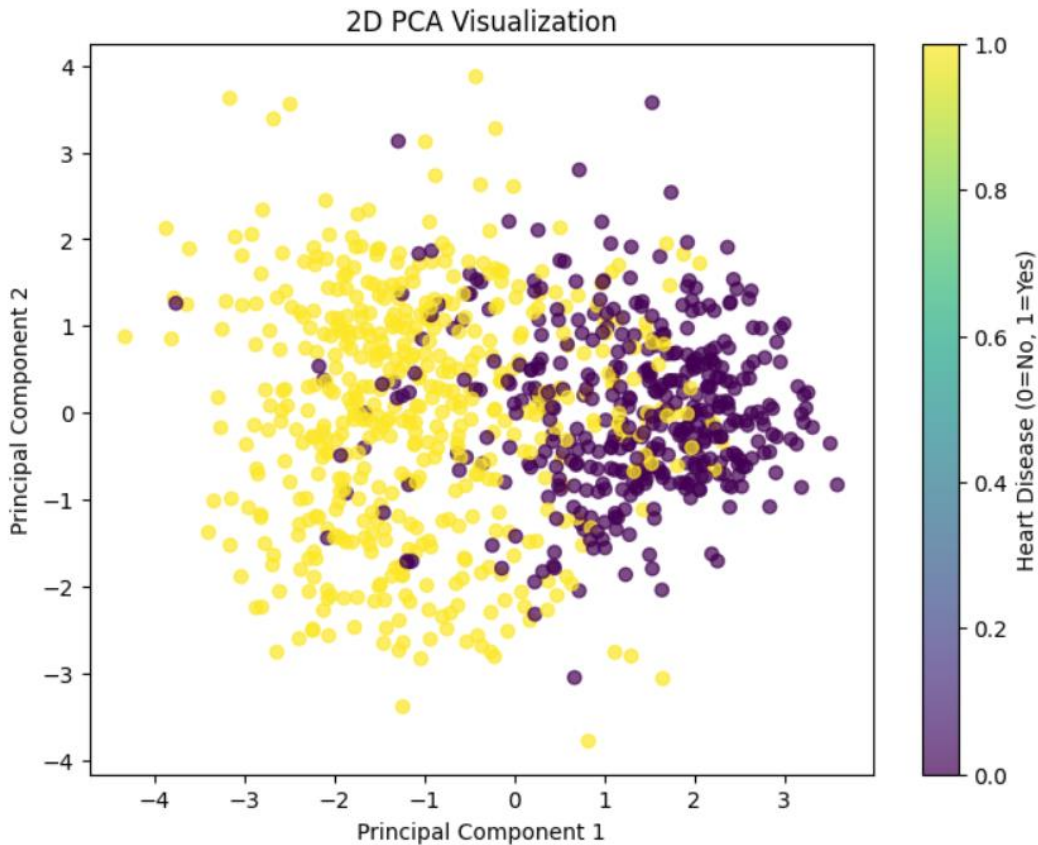
3.3 Feature Scaling

Machine learning algorithms, especially those that rely on distance-based metrics, perform better when features are on a similar scale. Feature scaling helps prevent features with larger numerical values from disproportionately influencing the model and improves convergence during training, ultimately leading to better model performance.

For this, continuous variables were scaled using **Standard Scaler**. Standardization ensures uniformity by transforming the data to have a mean of 0 and a standard deviation of 1.

PCA

PCA was conducted to reduce the dimensionality of the dataset to help visualize how the data points corresponding to the presence and absence of heart disease are distributed.



There is a moderate degree of overlap between the two classes, suggesting that PCA has captured some of the variance related to the target variable, but the separation is not distinct. Thus, PCA did not achieve clear class separability in this dataset.

Model Development

The data was split into training and testing sets using an 80-20 split. Models were trained using cross-validation to ensure robustness.

Model Training

Several machine learning algorithms were considered, including:

- Naïve Bayes
- Support Vector Machines (SVM)
- K-Nearest Neighbors (KNN)
- Decision Trees.

Support Vector Machine

```
# Grids hyperparameter
svm_param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': [0.1, 0.01, 0.001]
}
```

Hyperparameter Grid

The `svm_param_grid` dictionary defines the range of hyperparameters to test for the SVM classifier:

C: Controls the regularization strength; a smaller value allows more misclassifications (soft margin), focusing on simplicity and generalization, while larger values reduce misclassification tolerance for a stricter model fit; values tested: [0.1, 1, 10].

Kernel: Specifies the kernel type, which defines the transformation of the input data to a higher-dimensional space; 'linear' applies no transformation and works well with linearly separable data, while 'rbf' maps data to a higher-dimensional space using Gaussian similarity.

Gamma: Determines the influence of a single training example for RBF kernels; a higher gamma value means each data point has a narrower influence, leading to a more complex model, while a lower gamma value results in a smoother decision boundary; values tested: [0.1, 0.01, 0.001].

```
# Set up GridSearchCV for SVM
svm_model = SVC(random_state=42)
svm_grid_search = GridSearchCV(estimator=svm_model, param_grid=svm_param_grid,
                                scoring='f1_weighted', cv=5, return_train_score=True)
```

- Initializes the SVM classifier with a fixed `random_state` for reproducibility.
- Default parameters will be overridden by the best combination identified in the grid search
- **estimator=svm_model:** Specifies the SVM model as the estimator.
- **param_grid=svm_param_grid:** Uses the hyperparameter grid defined earlier.
- **scoring='f1_weighted':** Optimizes the model based on the weighted F1-score, balancing precision and recall for imbalanced datasets.
- **cv=5:** Performs 5-fold cross-validation to ensure robustness of results.
- **return_train_score=True:** Stores training scores for analysis, useful for evaluating overfitting or underfitting.

SVM grid search fitting

```
svm_grid_search.fit(X_train, y_train)
```

```
GridSearchCV
GridSearchCV(cv=5, estimator=SVC(random_state=42),
  param_grid={'C': [0.1, 1, 10], 'gamma': [0.1, 0.01, 0.001],
    'kernel': ['linear', 'rbf']},
  return_train_score=True, scoring='f1_weighted')
  best_estimator_: SVC
    SVC(C=1, gamma=0.1, random_state=42)
      SVC
        SVC(C=1, gamma=0.1, random_state=42)
```

Trains the SVM classifier on the training data (X_train and y_train) using all combinations of hyperparameters in svm_param_grid.

For each combination, cross-validation is applied, and the best combination is selected based on the weighted F1-score.

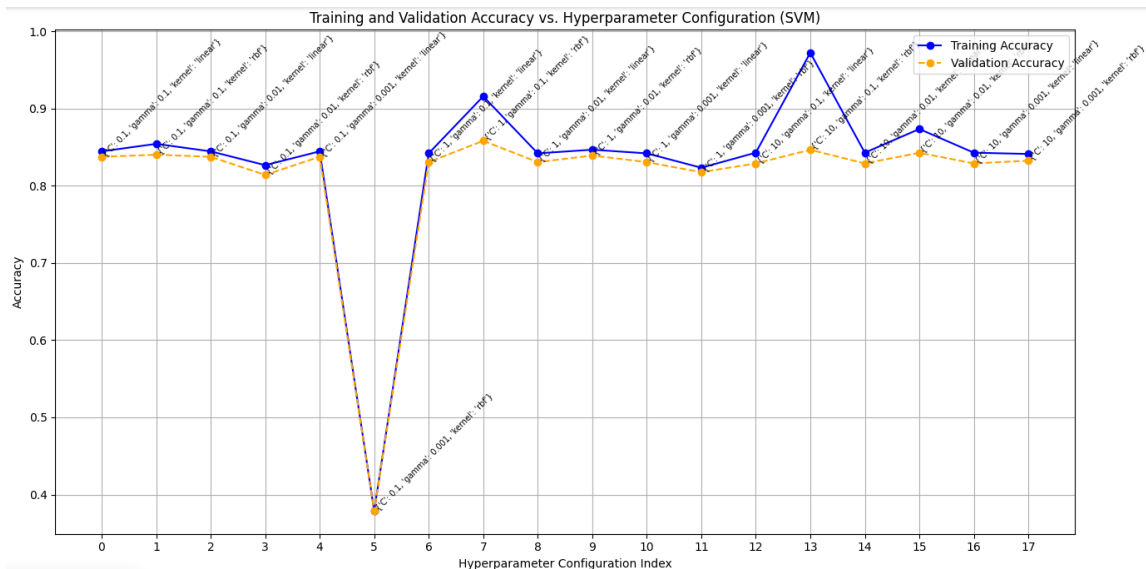
Best hyperparameters

```
print("Best parameters for SVM:", svm_grid_search.best_params_)
best_svm_model = svm_grid_search.best_estimator_
y_val_pred_svm = best_svm_model.predict(X_val)
```

```
Best parameters for SVM: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
```

Training & Validation accuracy vs hyperparameters

Retrieves the trained SVM model with the best combination of hyperparameters.



The grid search results aligns perfectly with point 7 that has high validation (~84%) and training accuracy (~91%) with minimal overfitting the graph shows no overfitting except in point 13 and a very clear underfitting at point 15 at validation and training accuracy of 35%.

Naive Bayes

```
# Define hyperparameter grids for Naive Bayes
nb_param_grid = {'var_smoothing': [1e-9, 1e-8, 1e-7, 1e-6]}
```

var_smoothing: A parameter that adds a small value to the variance of each feature, helping stabilize computations and prevent division by zero in cases where a feature's variance is extremely small or zero; smaller values (1e-9) lead to less smoothing, which can fit the data more tightly, while larger values (1e-6) add more smoothing, helping generalization, especially for small or noisy datasets.

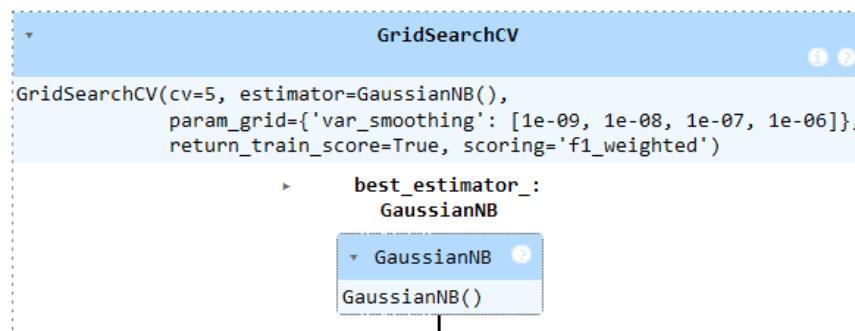
```
# Set up GridSearchCV for Naive Bayes
nb_model = GaussianNB()
nb_grid_search = GridSearchCV(estimator=nb_model, param_grid=nb_param_grid,
                                scoring='f1_weighted', cv=5, return_train_score=True)
```

Initializes a Gaussian Bayes classifier, assuming features are normally distributed.

- **estimator=nb_model:** Specifies the Gaussian Naive Bayes model to be tuned.
- **param_grid=nb_param_grid:** Passes the hyperparameter grid for var_smoothing.
- **scoring='f1_weighted':** Optimizes the model for the weighted F1-score, which is suitable for imbalanced datasets.
- **cv=5:** Performs 5-fold cross-validation to ensure robust evaluation by splitting the data into training and validation sets multiple times.
- **return_train_score=True:** Stores training scores to analyze whether the model is overfitting or underfitting.

Bayes classifier grid search fitting

```
nb_grid_search.fit(X_train, y_train)
```



Fits the model to the training data (X_train and y_train) using all combinations of var_smoothing specified in the grid.

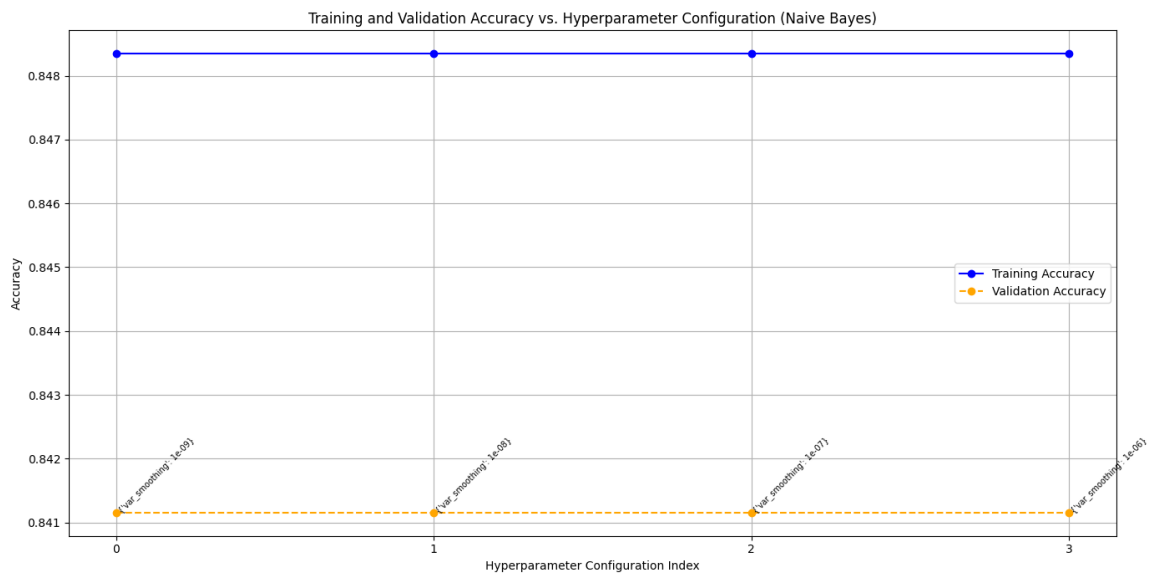
For each combination, 5-fold cross-validation is performed to evaluate the F1-score, and the best parameter is selected.

Best hyperparameters

```
print("Best parameters for Naïve Bayes:", nb_grid_search.best_params_)
best_nb_model = nb_grid_search.best_estimator_
y_val_pred_nb = best_nb_model.predict(X_val)
```

```
Best parameters for Naïve Bayes: {'var_smoothing': 1e-09}
```

Retrieves the trained Bayes model with the best combination of hyperparameters.



The grid search results align with point 0 that has high validation (~84.1%) and training accuracy(~84.8%) with minimal overfitting and difference between them with a minimal extra generalization as not very needed the whole graph shows no overfitting in the whole hyperparameters.

Knn Classifier

```
# Define the parameter grid
knn_param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
```

Hyperparameter Grid

`knn_param_grid` specifies the hyperparameters and their possible values:

n_neighbors: The number of nearest neighbors to consider when classifying a data point; smaller values lead to models sensitive to local variations (risk of overfitting), while larger values make predictions smoother but may underfit the data; values tested: [3, 5, 7, 9, 11].

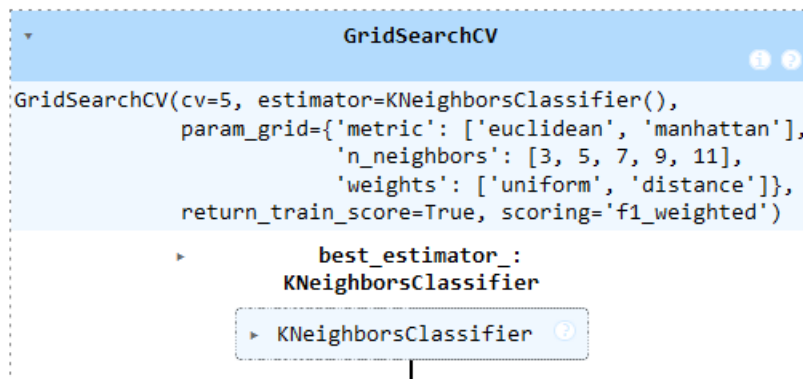
weights: Defines the weight assigned to neighbors in classification; 'uniform' means all neighbors contribute equally, while 'distance' gives closer neighbors greater influence.

metric: Specifies the distance metric to calculate the proximity between points; 'euclidean' is the straight-line distance between points, while 'manhattan' is the sum of absolute differences along dimensions.

```
knn_model = KNeighborsClassifier()
knn_grid_search = GridSearchCV(estimator=knn_model, param_grid=knn_param_grid,
                                scoring='f1_weighted', cv=5, return_train_score=True)
```

KNN grid search fitting

```
knn_grid_search.fit(X_train, y_train)
```



Creates a default KNN model without specifying any parameters.

estimator=knn_model: Specifies the KNN classifier as the model to tune.

param_grid=knn_param_grid: Passes the hyperparameter grid to test.

scoring='f1_weighted': Evaluates models based on the weighted F1-score, which accounts for class imbalance.

cv=5: Performs 5-fold cross-validation to split the data into training and validation sets multiple times.

return_train_score=True: Stores training scores for analysis, helping assess overfitting or underfitting.

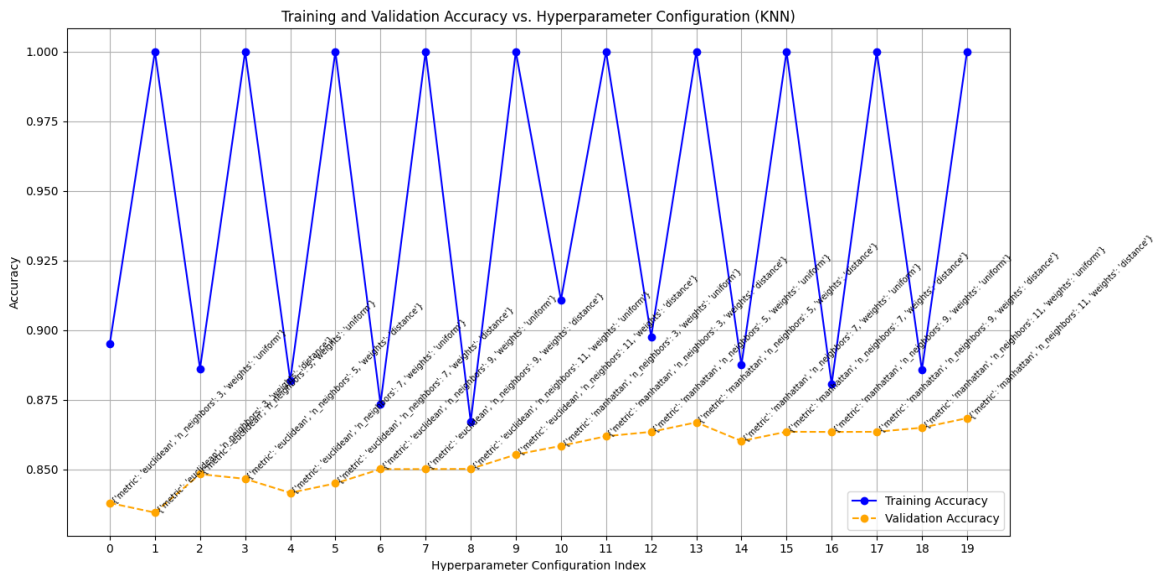
Fits the KNN classifier to the training data (X_{train} , y_{train}) using all possible combinations of hyperparameters from `knn_param_grid`.

For each combination, 5-fold cross-validation is performed, and the model's performance is evaluated based on the weighted F1-score.

Best hyperparameters

```
print("Best parameters for KNN:", knn_grid_search.best_params_)
best_knn_model = knn_grid_search.best_estimator_
```

Best parameters for KNN: {'metric': 'manhattan', 'n_neighbors': 11, 'weights': 'distance'}



Form the following graph a very clear overfitting occurrence at this graph showing training and validation accuracy for different KNN hyperparameter configurations

- The training accuracy (blue line) shows extreme peaks reaching 1.0 across configurations
- The validation accuracy (orange dashed line) stays much lower, around 0.85-0.87
- This large gap between training and validation performance is a classic sign of overfitting.

1. Validation Performance

- The validation accuracy shows a slight upward trend as the hyperparameter configuration index increases
- It starts around 0.83-0.84 and gradually improves to about 0.87

- The improvement is relatively modest compared to the fluctuations in training accuracy

2. Training Performance Pattern

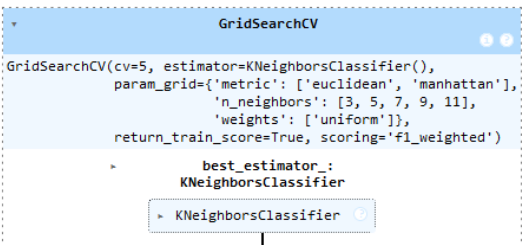
- The training accuracy shows a consistent zigzag pattern
- It alternates between perfect accuracy (1.0) and lower values around 0.88-0.90
- This suggests certain configurations are memorizing the training data while others maintain some generalization
- Around index 0: Training accuracy ~0.89 and validation ~0.83
- Around index 2: Training accuracy ~0.88 and validation ~0.85
- Around index 4, 6, 8, :Similar patterns where the training accuracy drops down closer to the validation accuracy

These points represent hyperparameter configurations where the model is generalizing better - it's not memorizing the training data (which would give perfect 1.0 training accuracy) but rather learning patterns that transfer better to unseen validation data. The smaller gap between training and validation performance at these points indicates better generalization.

But this isn't what Grid-Search results in as the model best hyperparameter produced as it only evaluates on training samples **was point 19** which had a training accuracy of 100% and validation accuracy of 86 % a clear overfitting so by observations we realized that the hyperparameter causing overfitting is the weights 'distance' so we removed it from the hyperparameters to be like this

```
# Define the parameter grid with higher n_neighbors values
knn_param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11],
    'weights': ['uniform'],
    'metric': ['euclidean', 'manhattan']
}

knn_model = KNeighborsClassifier()
knn_grid_search = GridSearchCV(estimator=knn_model, param_grid=knn_param_grid, scoring='f1_weighted', cv=5, return_train_score=True)
knn_grid_search.fit(X_train, y_train)
```



The screenshot shows a Jupyter Notebook cell with the following code and output:

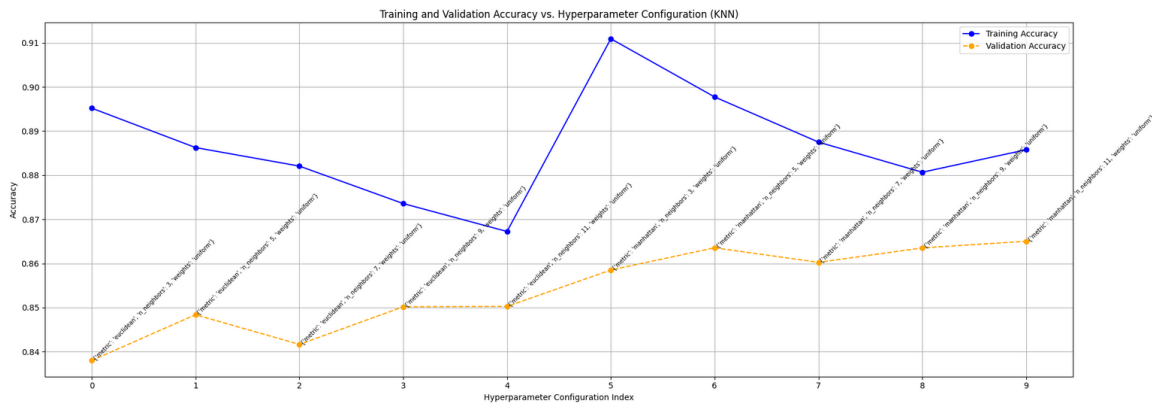
```
GridSearchCV(cv=5, estimator=KNeighborsClassifier(),
             param_grid={'metric': ['euclidean', 'manhattan'],
                         'n_neighbors': [3, 5, 7, 9, 11],
                         'weights': ['uniform']},
             return_train_score=True, scoring='f1_weighted')
```

The output shows the best estimator is a `KNeighborsClassifier` with the following parameters:

```
best_estimator_:
  KNeighborsClassifier
    n_neighbors: 3
    metric: euclidean
    weights: uniform
```

```
print("Best parameters for KNN:", knn_grid_search.best_params_)
best_knn_model = knn_grid_search.best_estimator_
```

Best parameters for KNN: {'metric': 'manhattan', 'n_neighbors': 11, 'weights': 'uniform'}



Overfitting got reduced in a very noticeable way here only point 6 might have a slight overfitting however it's not selected by grid-search **point 9 was selected** where training accuracy ~ 88.6% and validation accuracy ~86.5%.

Decision Tree

Hyper-Parameters

```
dt_param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 4, 5],
    'min_samples_split': [35, 45],
    'min_samples_leaf': [15, 20],
    'ccp_alpha': [0.02, 0.04]
}
```

- **criterion:** 'gini' uses the Gini Impurity to evaluate splits, while 'entropy' uses information gain (Entropy).
- **max_depth:** Restricts tree growth to prevent overfitting.
- **min_samples_split and min_samples_leaf:** Control the minimum number of samples required to create splits or leaf nodes, reducing the model's complexity.
- **ccp_alpha:** Cost Complexity Pruning parameter that controls pruning by balancing accuracy and simplicity; options: [0.02, 0.04]; larger values result in more aggressive pruning, potentially underfitting the data, while smaller values retain more of the tree structure, risking overfitting; controls the trade-off between model complexity and accuracy.

```
dt_model = DecisionTreeClassifier(random_state = 42)
dt_grid_search = GridSearchCV( estimator=dt_model, param_grid=dt_param_grid, scoring='accuracy', cv=5, return_train_score=True, n_jobs=-1, # Use all CPU cores
                               verbose=2 # Track progress
)

dt_grid_search.fit(X_train, y_train)

Fitting 5 folds for each of 48 candidates, totalling 240 fits
```

```
GridSearchCV
GridSearchCV(cv=5, estimator=DecisionTreeClassifier(random_state=42), n_jobs=-1,
             param_grid={'ccp_alpha': [0.02, 0.04],
                         'criterion': ['gini', 'entropy'],
                         'max_depth': [3, 4, 5], 'min_samples_leaf': [15, 20],
                         'min_samples_split': [35, 45]},
             return_train_score=True, scoring='accuracy', verbose=2)
  > best_estimator_:
      DecisionTreeClassifier
    > DecisionTreeClassifier
```

Initializes a Decision Tree with a fixed random state for reproducibility.

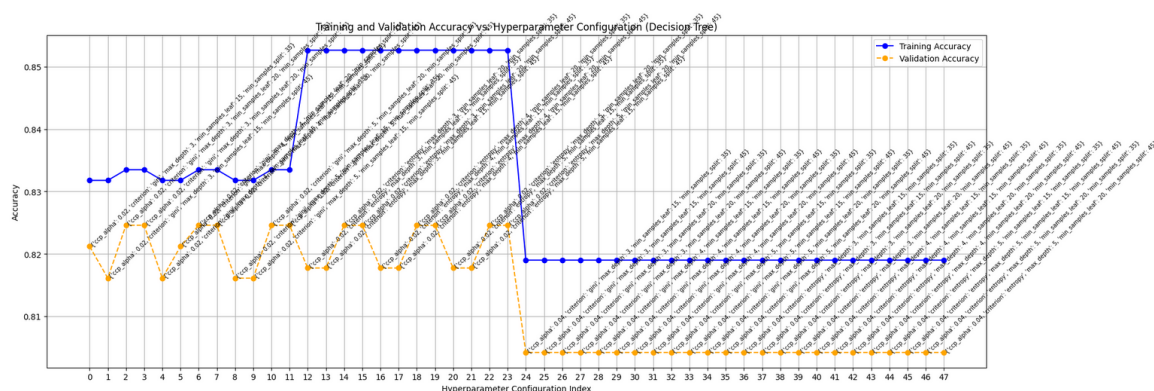
- **param_grid**: Provides the grid of hyperparameters to explore.
- **scoring='accuracy'**: Optimizes for accuracy during cross-validation.
- **cv=5**: Uses 5-fold cross-validation for reliable evaluation.
- **n_jobs=-1**: Utilizes all available CPU cores for faster computation.
- **verbose=2**: Displays detailed logs for monitoring progress.

Searches through all parameter combinations defined in dt_param_grid to identify the best performing configuration using 5-fold cross-validation.

```
print("Best parameters for Decision Tree:", dt_grid_search.best_params_)
best_dt_model = dt_grid_search.best_estimator_
```

Extracts and displays the best combination of hyperparameters and assigns the tuned model to best_dt_model.

```
Best parameters for Decision Tree: {'ccp_alpha': 0.02, 'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 20, 'min_samples_split': 35}
```



The graph shows three distinct regions:

1. Indices 0-11: Best performance with balanced metrics
 - Training: ~0.833
 - Validation: ~0.822

- Small gap: ~ 0.011
2. Indices 12-23: Overfitting region
 - Training: ~ 0.854
 - Validation: ~ 0.822
 - Large gap: ~ 0.032
 3. Indices 24+: Underfitting region
 - Both metrics around 0.82 and 0.805

Based on this, the optimal parameters would be those producing the results in the first region.

we haven't completely eliminated overfitting (which is often impossible and sometimes undesirable), we've reduced it to a very manageable level. This slight difference between training and validation performance is often expected and acceptable in practice, as long as the model generalizes well to new data.

It's selected by grid-search **point 2 was selected** where training accuracy $\sim 83.4\%$ and validation accuracy $\sim 82.5\%$.

Model Evaluation

Evaluation metrics included:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Evaluation functions

plot_confusion_matrix function

```
def plot_confusion_matrix(y_true, y_pred, model_name):  
    cm = confusion_matrix(y_true, y_pred)  
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_true))  
    disp.plot(cmap='Blues')  
    plt.title(f"Confusion Matrix for {model_name}")  
    plt.show()
```

This function generates and visualizes a confusion matrix, which is a table that summarizes the performance of a classification model by showing the counts of true positives, true negatives, false positives, and false negatives.

Parameters:

- `y_true`: The actual (true) labels for the data points.

- `y_pred`: The predicted labels generated by the model.
 - `model_name`: A string indicating the name of the model for labeling plot.
1. **Generate Confusion Matrix:** `confusion_matrix(y_true, y_pred)` calculates the confusion matrix using the true and predicted labels, outputting a 2D array where rows represent the actual classes and columns represent the predicted classes.
 2. **Create a Display Object:** `ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_true))` creates a display object for the confusion matrix with the unique labels from `y_true`.
 3. **Plot the Matrix:** `disp.plot(cmap='Blues')` visualizes the confusion matrix using a blue color map for clarity; `plt.title(f"Confusion Matrix for {model_name}")` adds a title to the plot for better context; display the plot.

This function helps interpret a classification model's predictions, allowing you to identify where the model is performing well and where it's making mistakes.

evaluate_model function

```
def evaluate_model(y_true, y_pred, model_name):
    print(f"\n{model_name} Performance:")
    results = classification_report(y_true, y_pred)
    print(results)
    print("Accuracy:", accuracy_score(y_true, y_pred))
```

This function evaluates the performance of a classification model by generating a detailed performance report and calculating its accuracy and provides both a detailed analysis by the classification report and a high-level summary (accuracy) of the model's performance. It's especially useful for comparing multiple models or fine-tuning a single model.

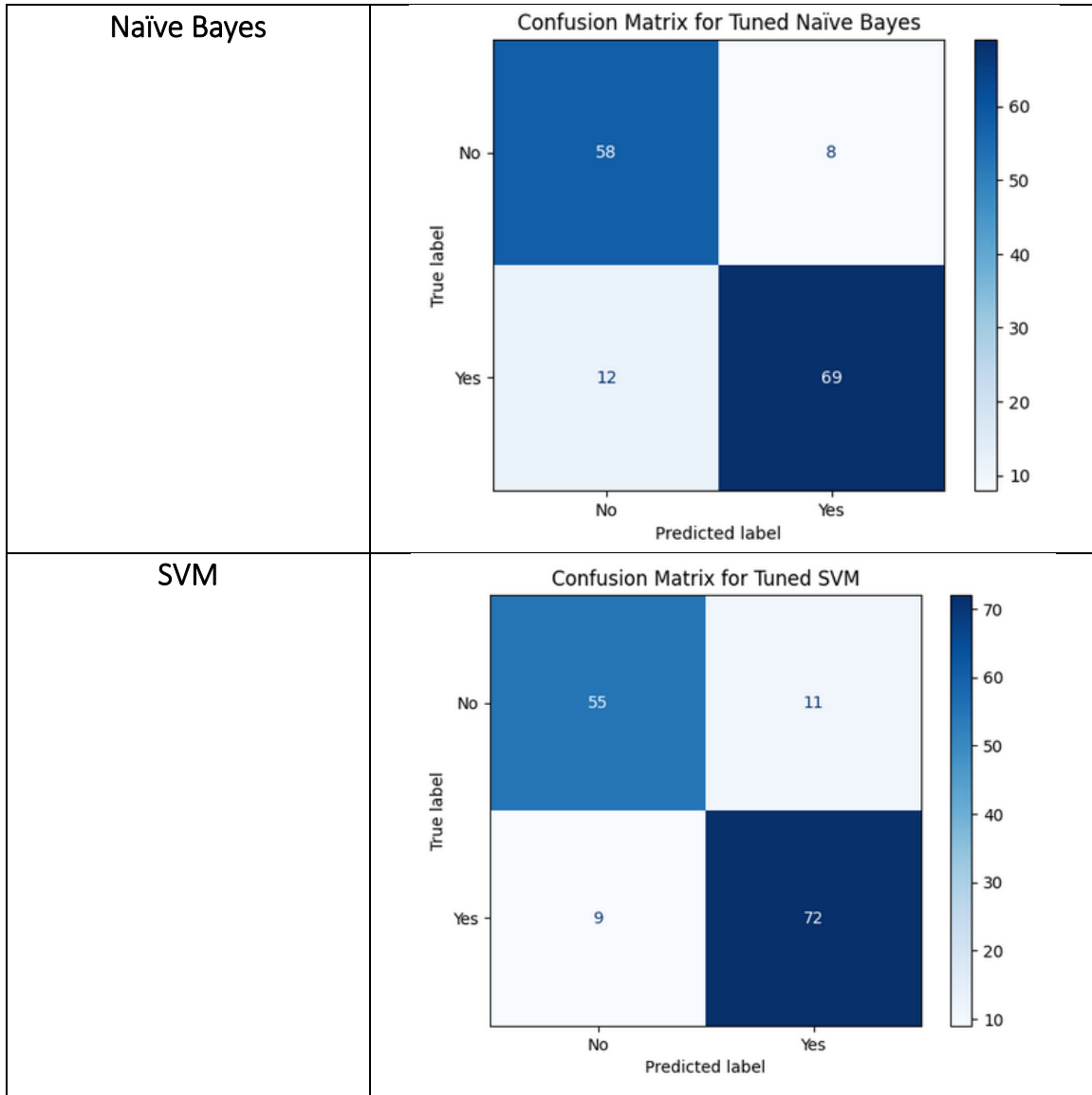
Parameters:

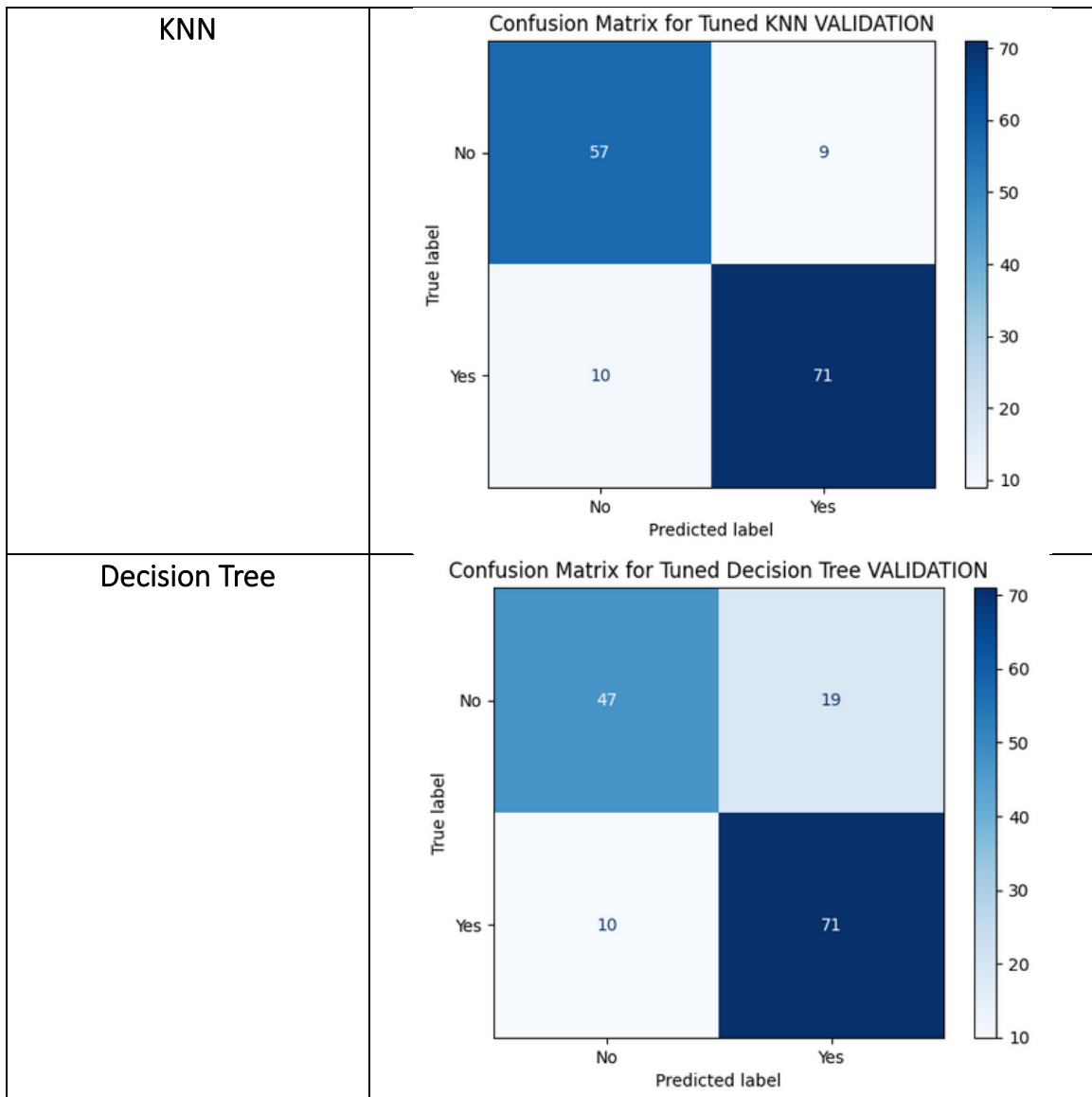
- `y_true`: The actual (true) labels for the data points.
 - `y_pred`: The predicted labels generated by the model.
 - `model_name`: A string indicating the name of the model used for reporting.
1. **Generate Classification Report:** `classification_report(y_true, y_pred)` creates a textual summary of precision, recall, F1-score, and support for each class; Precision measures how many predicted positive instances are actually positive, Recall measures how many actual positive instances were correctly predicted, F1-score is the harmonic mean of precision and recall, and Support is the number of true instances for each class.
 2. **Print the Classification Report.**

3. **Calculate Accuracy:** `accuracy_score(y_true, y_pred)` computes the overall accuracy as the ratio of correct predictions to total predictions and prints the accuracy value to summarize the model's performance.

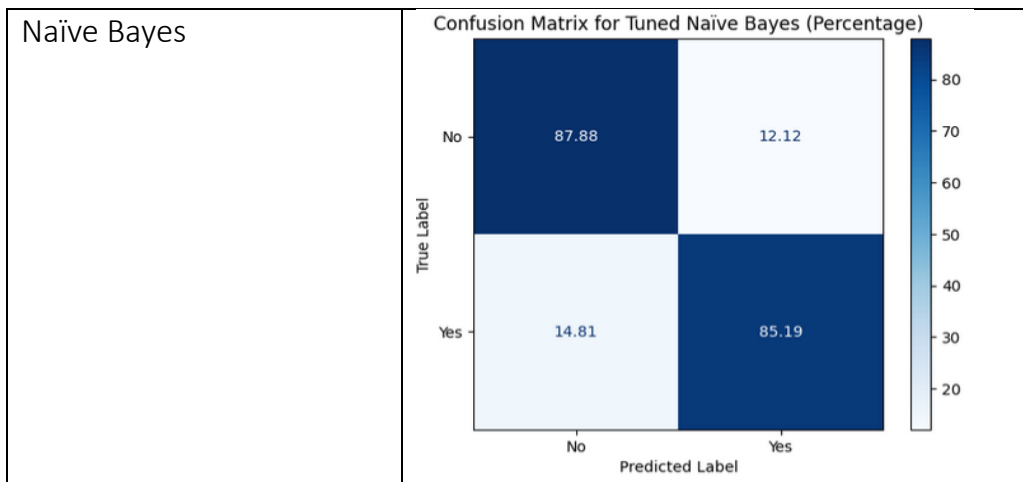
Comparison between 4 Classifiers

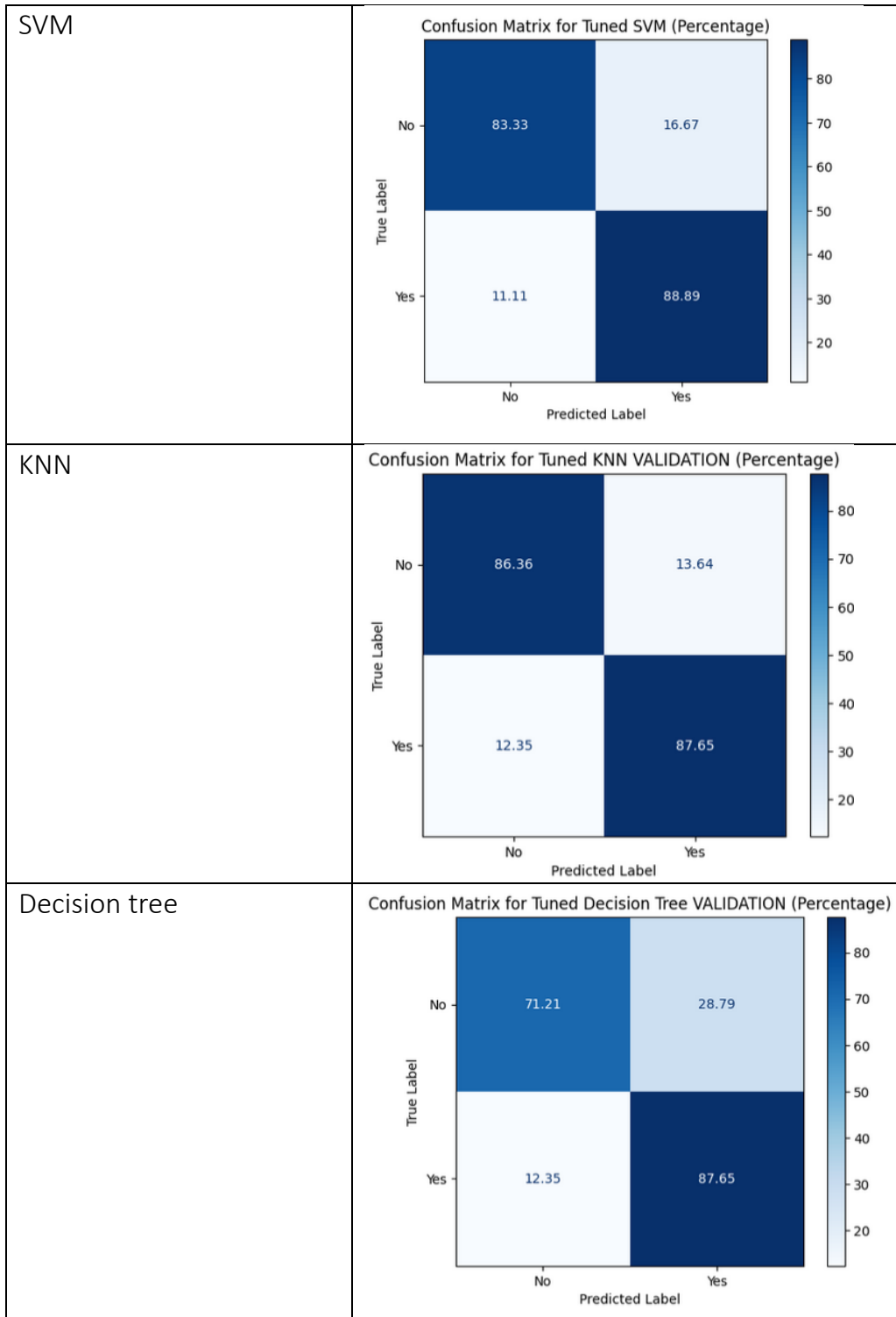
Validation Confusion Matrix





Confusion matrix in %





Accuracy

- Naive Bayes: $(58 + 69)/(58 + 8 + 12 + 69) = 86.4\%$

- SVM: $(55 + 72)/(55 + 11 + 9 + 72) = 86.4\%$

- KNN: $(57 + 71)/(57 + 9 + 10 + 71) = 87.1\%$
- Decision Tree: $(47 + 71)/(47 + 19 + 10 + 71) = 80.3\%$

True Positives (TP) / False Negatives (FN):

- Naive Bayes: 69 TP, 12 FN
- SVM: 72 TP, 9 FN
- KNN: 71 TP, 10 FN
- Decision Tree: 71 TP, 10 FN

True Negatives (TN) / False Positives (FP):

- Naive Bayes: 58 TN, 8 FP
- SVM: 55 TN, 11 FP
- KNN: 57 TN, 9 FP
- Decision Tree: 47 TN, 19 FP

Summery

We can conclude that:

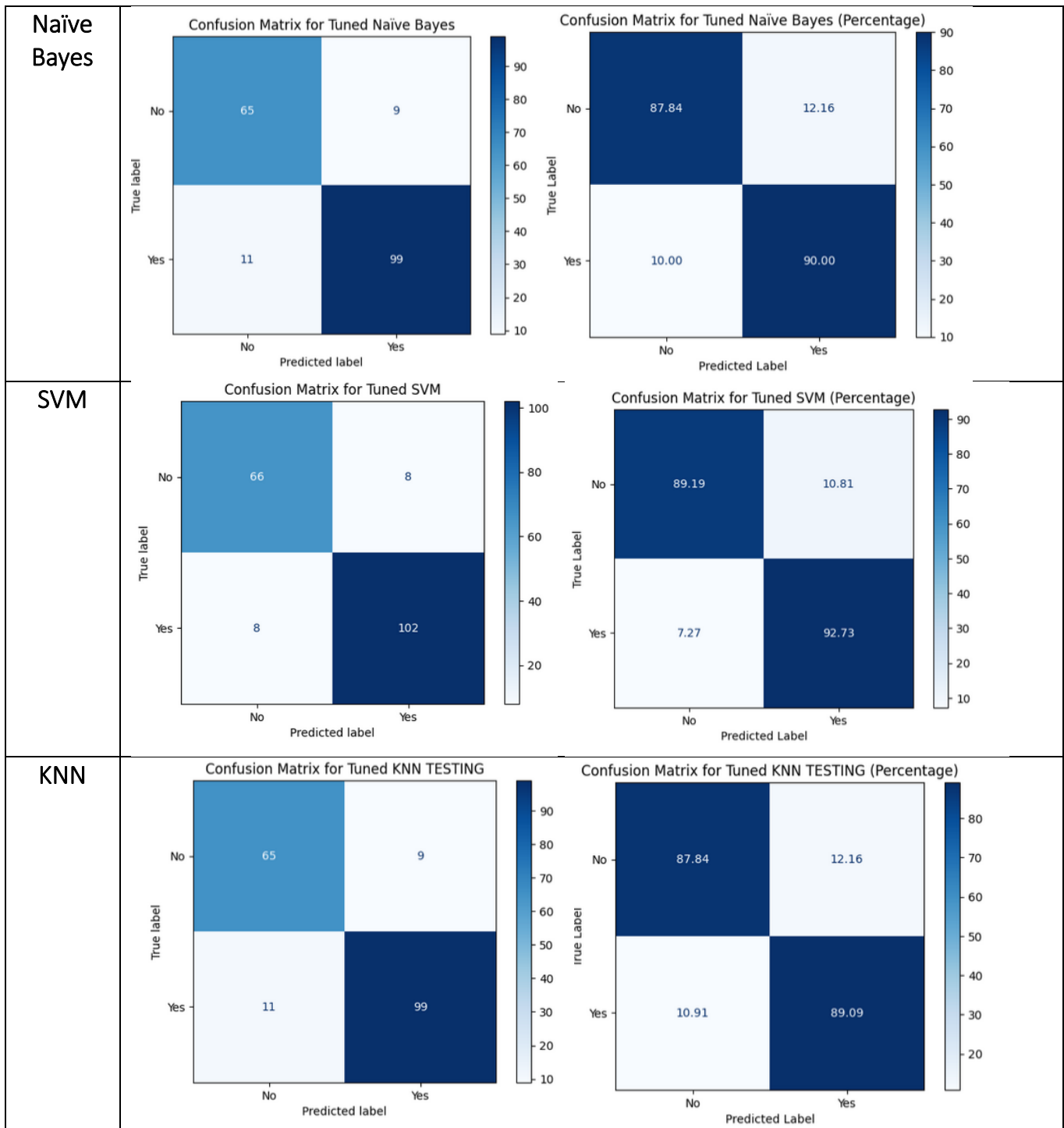
1. KNN performs slightly better overall with the highest accuracy (87.1%)
2. SVM has the highest number of true positives (72%)
3. Naive Bayes has the highest number of true negatives (58%)
4. Decision Tree shows the most false positives (19%), making it the least precise model

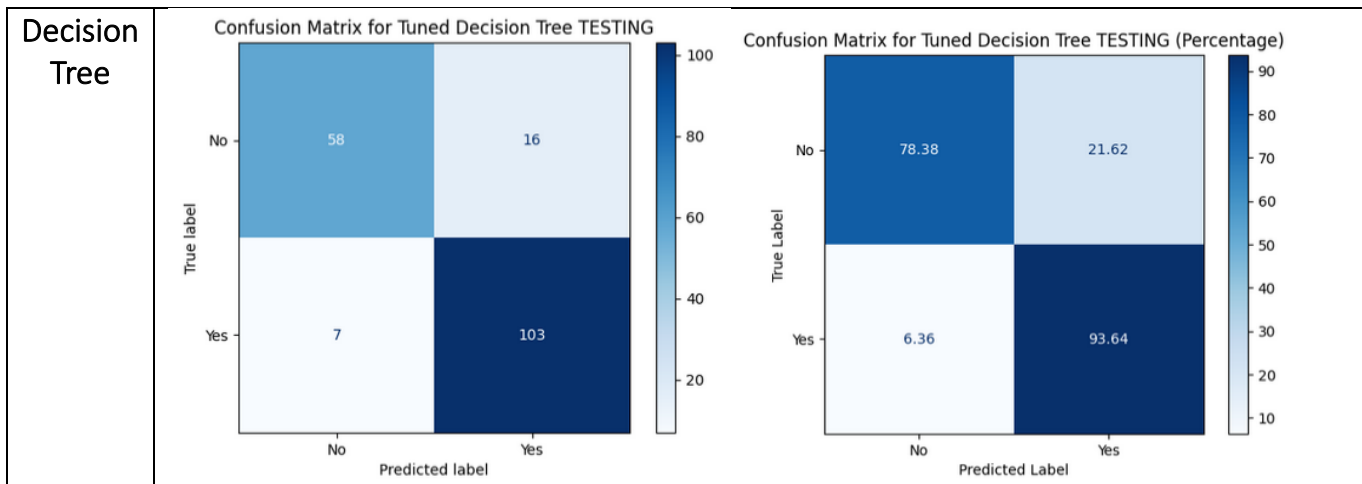
Overall performance

1. KNN appears to be the most balanced and best performing model
2. SVM and Naive Bayes are very close in performance
3. Decision Tree lags behind the others, particularly in handling negative cases

The first three models (Naive Bayes, SVM, and KNN) perform quite similarly, with accuracies within 1% of each other. The Decision Tree model shows notably worse performance, particularly in terms of false positives, which could make it less suitable if minimizing false positives is important for your use case.

Testing Confusion Matrix





Accuracy

- Naive Bayes: $(65 + 99)/(65 + 9 + 11 + 99) = 89.1\%$
- SVM: $(66 + 102)/(66 + 8 + 8 + 102) = 91.3\%$
- KNN: $(65 + 99)/(65 + 9 + 11 + 99) = 89.1\%$
- Decision Tree: $(58 + 103)/(58 + 16 + 7 + 103) = 87.5\%$

True Positives (TP) / False Negatives (FN)

- Naive Bayes: 99 TP, 11 FN
- SVM: 102 TP, 8 FN
- KNN: 99 TP, 11 FN
- Decision Tree: 103 TP, 7 FN

True Negatives (TN) / False Positives (FP)

- Naive Bayes: 65 TN, 9 FP
- SVM: 66 TN, 8 FP
- KNN: 65 TN, 9 FP
- Decision Tree: 58 TN, 16 FP

Summery

1. SVM performs the best overall with 91.3% accuracy
2. Naive Bayes and KNN show identical performance (89.1%)
3. Decision Tree has the highest number of true positives (103) but also the most false positives (16) 87.5% accuracy

4. SVM shows the most balanced performance with high accuracy and low error rates in both classes

Comparing models

1. SVM emerges as the clear winner with the best overall metrics
2. Naive Bayes and KNN perform identically, slightly behind SVM
3. Decision Tree shows good performance on positive cases but struggles with false positives
4. SVM achieves the best balance between minimizing both false positives and false negatives

SVM showing clear superiority in overall performance. The Decision Tree model continues to struggle with false positives, though it shows strong performance in identifying positive cases.

Naive Bayes

- Validation: 86.4% accuracy $(58+69)/(147)$
- Testing: 89.1% accuracy $(65+99)/(184)$
- Performance improved in testing
- Maintained consistent balance between false positives and negatives

SVM

- Validation: 86.4% accuracy $(55+72)/(147)$
- Testing: 91.3% accuracy $(66+102)/(184)$
- Significant improvement in testing
- Most stable and best performing model across both sets
- Improved true positive rate while maintaining low false positives

KNN

- Validation: 87.1% accuracy $(57+71)/(147)$
- Testing: 89.1% accuracy $(65+99)/(184)$
- Slight improvement in testing
- Very consistent performance between sets

- Shows good generalization ability

Decision Tree

- Validation: 80.3% accuracy (47+71)/(147)
- Testing: 87.5% accuracy (58+103)/(184)
- Largest improvement from validation to testing
- However, still shows highest false positive rate in both sets
- Most variable performance between validation and testing

Results

1. All models performed better on the test set than the validation set
2. SVM showed the most robust performance across both sets
3. Decision Tree showed the largest improvement but remained the least precise
4. KNN and Naive Bayes maintained very consistent performance patterns

The improved test set performance suggests that:

- The models generalized well to new data

SVM emerges as the best choice as it:

- Showed highest accuracy in testing
- Maintained consistent improvement from validation to testing
- Had the best balance of precision and recall across both sets

Validation Classification Report

Naïve Bayes	Tuned Naïve Bayes Performance:				
		precision	recall	f1-score	support
	No	0.83	0.88	0.85	66
	Yes	0.90	0.85	0.87	81
	accuracy			0.86	147
	macro avg	0.86	0.87	0.86	147
	weighted avg	0.87	0.86	0.86	147
	Accuracy: 0.8639455782312925				

SVM	Tuned SVM Performance: <table><tr><td></td><td>precision</td><td>recall</td><td>f1-score</td><td>support</td></tr><tr><td>No</td><td>0.86</td><td>0.83</td><td>0.85</td><td>66</td></tr><tr><td>Yes</td><td>0.87</td><td>0.89</td><td>0.88</td><td>81</td></tr><tr><td>accuracy</td><td></td><td></td><td>0.86</td><td>147</td></tr><tr><td>macro avg</td><td>0.86</td><td>0.86</td><td>0.86</td><td>147</td></tr><tr><td>weighted avg</td><td>0.86</td><td>0.86</td><td>0.86</td><td>147</td></tr></table> Accuracy: 0.8639455782312925		precision	recall	f1-score	support	No	0.86	0.83	0.85	66	Yes	0.87	0.89	0.88	81	accuracy			0.86	147	macro avg	0.86	0.86	0.86	147	weighted avg	0.86	0.86	0.86	147
	precision	recall	f1-score	support																											
No	0.86	0.83	0.85	66																											
Yes	0.87	0.89	0.88	81																											
accuracy			0.86	147																											
macro avg	0.86	0.86	0.86	147																											
weighted avg	0.86	0.86	0.86	147																											
KNN	Tuned KNN Performance: <table><tr><td></td><td>precision</td><td>recall</td><td>f1-score</td><td>support</td></tr><tr><td>No</td><td>0.83</td><td>0.88</td><td>0.85</td><td>66</td></tr><tr><td>Yes</td><td>0.90</td><td>0.85</td><td>0.87</td><td>81</td></tr><tr><td>accuracy</td><td></td><td></td><td>0.86</td><td>147</td></tr><tr><td>macro avg</td><td>0.86</td><td>0.87</td><td>0.86</td><td>147</td></tr><tr><td>weighted avg</td><td>0.87</td><td>0.86</td><td>0.86</td><td>147</td></tr></table> Accuracy: 0.8639455782312925		precision	recall	f1-score	support	No	0.83	0.88	0.85	66	Yes	0.90	0.85	0.87	81	accuracy			0.86	147	macro avg	0.86	0.87	0.86	147	weighted avg	0.87	0.86	0.86	147
	precision	recall	f1-score	support																											
No	0.83	0.88	0.85	66																											
Yes	0.90	0.85	0.87	81																											
accuracy			0.86	147																											
macro avg	0.86	0.87	0.86	147																											
weighted avg	0.87	0.86	0.86	147																											
Decision Tree	Tuned Decision Tree Performance: <table><tr><td></td><td>precision</td><td>recall</td><td>f1-score</td><td>support</td></tr><tr><td>No</td><td>0.82</td><td>0.71</td><td>0.76</td><td>66</td></tr><tr><td>Yes</td><td>0.79</td><td>0.88</td><td>0.83</td><td>81</td></tr><tr><td>accuracy</td><td></td><td></td><td>0.80</td><td>147</td></tr><tr><td>macro avg</td><td>0.81</td><td>0.79</td><td>0.80</td><td>147</td></tr><tr><td>weighted avg</td><td>0.80</td><td>0.80</td><td>0.80</td><td>147</td></tr></table> Accuracy: 0.8027210884353742		precision	recall	f1-score	support	No	0.82	0.71	0.76	66	Yes	0.79	0.88	0.83	81	accuracy			0.80	147	macro avg	0.81	0.79	0.80	147	weighted avg	0.80	0.80	0.80	147
	precision	recall	f1-score	support																											
No	0.82	0.71	0.76	66																											
Yes	0.79	0.88	0.83	81																											
accuracy			0.80	147																											
macro avg	0.81	0.79	0.80	147																											
weighted avg	0.80	0.80	0.80	147																											

Naive Bayes

- Accuracy: 86.39%
- For "No" class: precision=0.83, recall=0.88, F1=0.85
- For "Yes" class: precision=0.90, recall=0.85, F1=0.87
- Very balanced performance between classes

SVM

- Accuracy: 86.39%
- For "No" class: precision=0.86, recall=0.83, F1=0.85
- For "Yes" class: precision=0.87, recall=0.89, F1=0.88

- Slightly better at handling "Yes" class

KNN

- Accuracy: 86.39%
- For "No" class: precision=0.83, recall=0.88, F1=0.85
- For "Yes" class: precision=0.90, recall=0.85, F1=0.87
- Almost identical performance to Naive Bayes

Decision Tree

- Accuracy: 80.27%
- For "No" class: precision=0.82, recall=0.71, F1=0.76
- For "Yes" class: precision=0.79, recall=0.88, F1=0.83
- Shows clear performance drop compared to other models

Key observations

1. Naive Bayes, SVM, and KNN show remarkably similar overall accuracy (86.39%)
2. SVM has the most balanced precision-recall trade-off across both classes
3. Decision Tree lags behind with lower accuracy and more imbalanced metrics
4. Support shows slightly imbalanced dataset (66 "No" vs 81 "Yes" samples)

Model-specific highlights

- SVM: Most consistent across all metrics
- Naive Bayes & KNN: Nearly identical performance patterns
- Decision Tree: Shows particular weakness in recall for "No" class (0.71)

All models except Decision Tree show strong and similar performance in validation, with very little to differentiate between them. The Decision Tree's lower performance suggests it might be less suitable for this classification task.

Testing Classification Report

Naïve Bayes

Tuned Naïve Bayes Performance:				
	precision	recall	f1-score	support
No	0.86	0.88	0.87	74
Yes	0.92	0.90	0.91	110
accuracy			0.89	184
macro avg	0.89	0.89	0.89	184
weighted avg	0.89	0.89	0.89	184
Accuracy: 0.8913043478260869				

SVM

Tuned SVM Performance:					
	precision	recall	f1-score	support	
No	0.89	0.89	0.89	74	
Yes	0.93	0.93	0.93	110	
accuracy			0.91	184	
macro avg	0.91	0.91	0.91	184	
weighted avg	0.91	0.91	0.91	184	
Accuracy: 0.9130434782608695					

KNN

Tuned knn Performance:					
	precision	recall	f1-score	support	
No	0.84	0.88	0.86	74	
Yes	0.92	0.89	0.90	110	
accuracy			0.89	184	
macro avg	0.88	0.88	0.88	184	
weighted avg	0.89	0.89	0.89	184	
Accuracy: 0.8858695652173914					

Decision Tree

```
.....

Tuned Decision Tree Performance:
      precision    recall  f1-score   support

     No         0.89      0.78      0.83         74
     Yes         0.87      0.94      0.90        110

 accuracy          0.88          184
 macro avg         0.88      0.86      0.87          184
weighted avg         0.88      0.88      0.87          184

Accuracy: 0.875
```

Naive Bayes

- Improved precision for both classes compared to validation
- "No" class: precision=0.86, recall=0.88, F1=0.87
- "Yes" class: precision=0.92, recall=0.90, F1=0.91
- Balanced macro and weighted averages (0.89)

SVM

- Best overall performance metrics
- "No" class: precision=0.89, recall=0.89, F1=0.89
- "Yes" class: precision=0.93, recall=0.93, F1=0.93
- Highest macro and weighted averages (0.91)
- Most balanced performance across all metrics

KNN Testing

- Similar to validation but slightly improved
- "No" class: precision=0.84, recall=0.88, F1=0.86
- "Yes" class: precision=0.92, recall=0.89, F1=0.90
- Good macro and weighted averages (0.88, 0.89)

Decision Tree Testing

- "No" class: precision=0.89, recall=0.78, F1=0.83
- "Yes" class: precision=0.87, recall=0.94, F1=0.90

- Shows imbalance between precision and recall
- Macro average (0.88) shows improvement from validation

Dataset characteristics in testing

- Larger test set (184 samples vs 147 in validation)
- More imbalanced class distribution (74 "No" vs 110 "Yes")

The testing metrics confirm SVM as the superior model, showing both the highest performance and most balanced metrics across all classes.

Histogram Analysis

```
# classification reports for validation
report_val_svm = classification_report(y_val, y_val_pred_svm, output_dict=True)
report_val_nb = classification_report(y_val, y_val_pred_nb, output_dict=True)
report_val_knn = classification_report(y_val, y_val_pred_knn, output_dict=True)
report_val_dt = classification_report(y_val, y_val_pred_dt, output_dict=True)

# classification reports for testing
report_test_svm = classification_report(y_test, y_test_pred_svm, output_dict=True)
report_test_nb = classification_report(y_test, y_test_pred_nb, output_dict=True)
report_test_knn = classification_report(y_test, y_test_pred_knn, output_dict=True)
report_test_dt = classification_report(y_test, y_test_pred_dt, output_dict=True)

# Extract weighted average metrics
metrics = ["precision", "recall", "f1-score"]

# Validation metrics
svm_val_metrics = [report_val_svm["weighted avg"][metric] for metric in metrics]
nb_val_metrics = [report_val_nb["weighted avg"][metric] for metric in metrics]
knn_val_metrics = [report_val_knn["weighted avg"][metric] for metric in metrics]
dt_val_metrics = [report_val_dt["weighted avg"][metric] for metric in metrics]

# Testing metrics
svm_test_metrics = [report_test_svm["weighted avg"][metric] for metric in metrics]
nb_test_metrics = [report_test_nb["weighted avg"][metric] for metric in metrics]
knn_test_metrics = [report_test_knn["weighted avg"][metric] for metric in metrics]
dt_test_metrics = [report_test_dt["weighted avg"][metric] for metric in metrics]

x_labels = ["Precision", "Recall", "F1-Score"]

x = np.arange(len(metrics))
width = 0.15

fig, ax = plt.subplots(2, 1, figsize=(12, 12))

# Validation dataset
ax[0].bar(x - 1.5*width, svm_val_metrics, width, label="SVM (Validation)", color='pink')
ax[0].bar(x - 0.5*width, nb_val_metrics, width, label="Naïve Bayes (Validation)", color='brown')
ax[0].bar(x + 0.5*width, knn_val_metrics, width, label="KNN (Validation)", color='green')
ax[0].bar(x + 1.5*width, dt_val_metrics, width, label="Decision Tree (Validation)", color='orange')

ax[0].set_ylabel("Score")
ax[0].set_title("Weighted Metrics - Validation Set")
ax[0].set_xticks(x)
ax[0].set_xticklabels(x_labels)
ax[0].legend(loc='best')
```

```

# Testing dataset
ax[1].bar(x - 1.5*width, svm_test_metrics, width, label="SVM (Testing)", color='blue')
ax[1].bar(x - 0.5*width, nb_test_metrics, width, label="Naïve Bayes (Testing)", color='cyan')
ax[1].bar(x + 0.5*width, knn_test_metrics, width, label="KNN (Testing)", color='purple')
ax[1].bar(x + 1.5*width, dt_test_metrics, width, label="Decision Tree (Testing)", color='red')

ax[1].set_ylabel("Score")
ax[1].set_title("Weighted Metrics - Testing Set")
ax[1].set_xticks(x)
ax[1].set_xticklabels(x_labels)
ax[1].legend(loc="best")

plt.tight_layout()
plt.show()

```

This code evaluates the performance of four classification models (SVM, Naïve Bayes, KNN, and Decision Tree) on both validation and testing datasets. It uses classification reports to extract the weighted average metrics: Precision, Recall, and F1-Score.

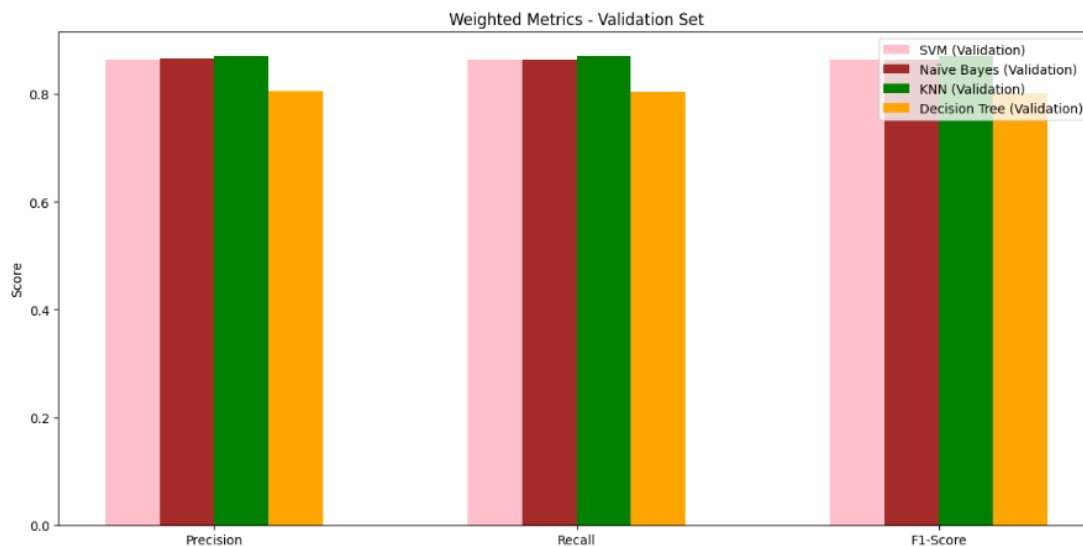
- Validation Metrics: Extracted from predictions on the validation set (y_val vs. y_val_pred_).
- Testing Metrics: Extracted from predictions on the testing set (y_test vs. y_test_pred_).

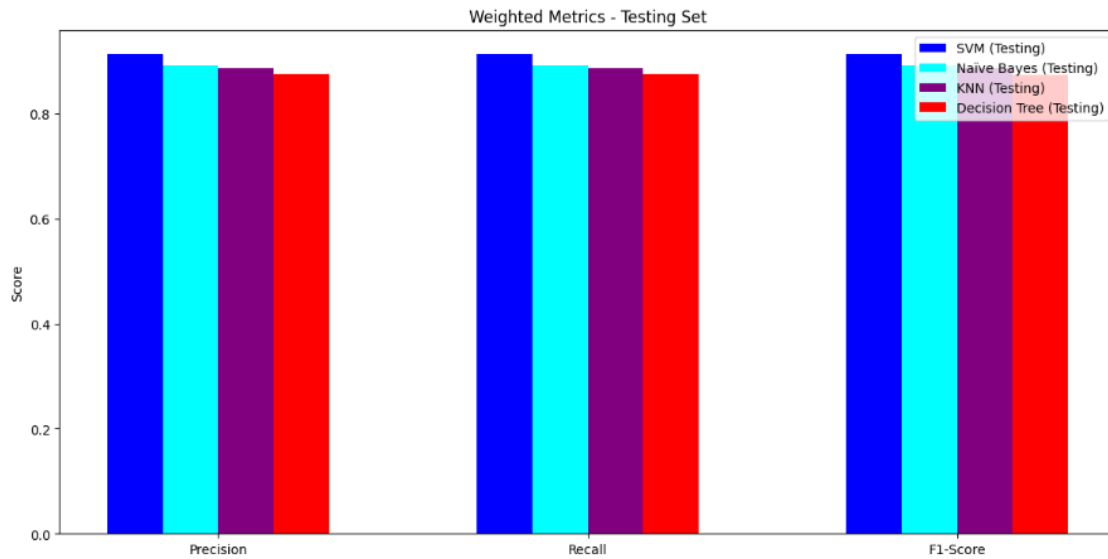
The results are visualized using bar plots:

Validation Plot: Displays weighted metrics for each model on the validation set.

Testing Plot: Displays weighted metrics for each model on the testing set.

Each bar represents a model's performance on a specific metric, providing an easy comparison across models.





Overall trend shows improved performance in testing compared to validation, with SVM emerging as the most robust and consistent performer across both sets.

Validation Set:

- All models except Decision Tree performed similarly (around 0.86-0.87)
- Decision Tree consistently lower (~0.80) across all metrics
- Very tight grouping between SVM, Naive Bayes, and KNN

Testing Set:

- SVM leads across all metrics (blue bars)
- Clearer separation between models
- Decision Tree (red bars) improved but still lowest performer
- All models showed better metrics compared to validation
- More pronounced differences in F1-Score

Clustering

Description of the plot_dendrogram Function

```
def plot_dendrogram(X, method):
    linkage_matrix = linkage(X, method=method)

    plt.figure(figsize=(10, 6))
    plt.title(f'Hierarchical Clustering Dendrogram ({method.capitalize()} Linkage)')
    plt.xlabel('Data Points or Clusters')
    plt.ylabel('Dissimilarity (Distance)')
    dendrogram(
        linkage_matrix,
        truncate_mode='level', # Truncate to simplify visualization
        p=5,                  # Show top 5 levels of hierarchy
        leaf_rotation=90,
        leaf_font_size=10,
        show_contracted=True  # Better readability for large datasets
    )
    plt.show()
    return linkage_matrix
```

The plot_dendrogram function visualizes the hierarchical clustering process for a given dataset (X) using a specified linkage method.

Inputs:

1. **X**: The dataset to be clustered, usually in a 2D array or DataFrame format.
2. **method**: The linkage method used for clustering. Examples include:
 - single: Minimum distance between clusters.
 - complete: Maximum distance between clusters.
 - average: Average distance between clusters.
 - ward: Minimizes the variance within clusters.
 - Cenroid: the distance between clusters is calculated as the Euclidean distance between their centroids

1. Calculate the Linkage Matrix:

This matrix encodes the distances and merges between clusters at each step of the hierarchy.

2. Create a Dendrogram:

The dendrogram is a tree-like visualization showing how individual data points or clusters merge step by step.

Parameters for the dendrogram:

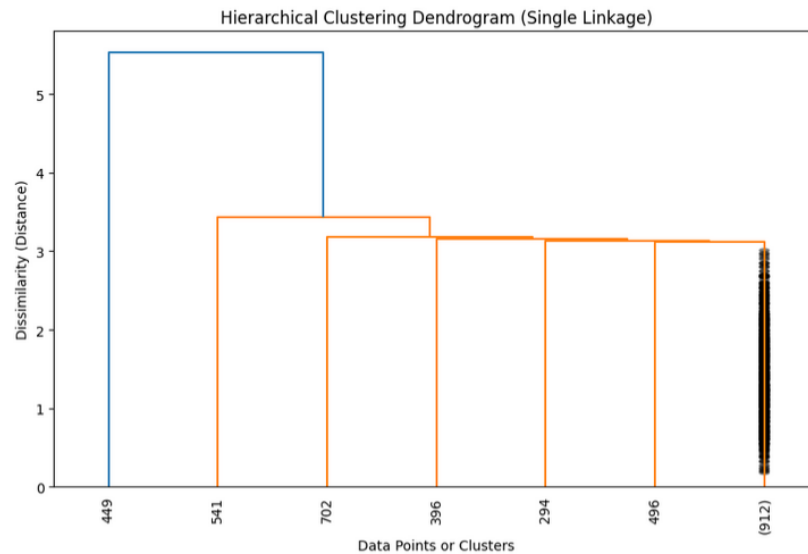
- truncate_mode='level': Limits the visualization to the top p levels of the hierarchy for simplicity.

- `p=5`: Displays only the top 5 levels to avoid clutter.
- `leaf_rotation=90`: Rotates labels on the x-axis for readability.
- `leaf_font_size=10`: Adjusts font size for leaf labels.
- `show_contracted=True`: Collapses intermediate nodes for a clearer view in large datasets.

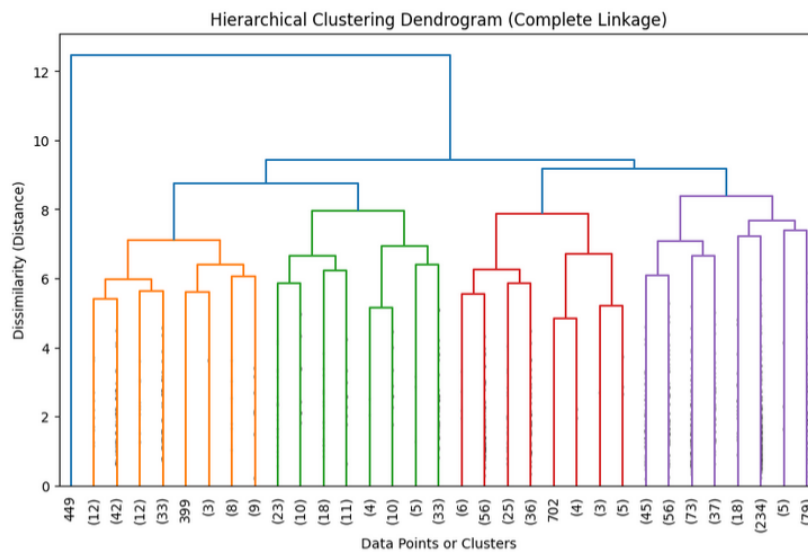
Returns the computed linkage matrix

Output: Provides insight how data points are grouped at different levels of dissimilarity.

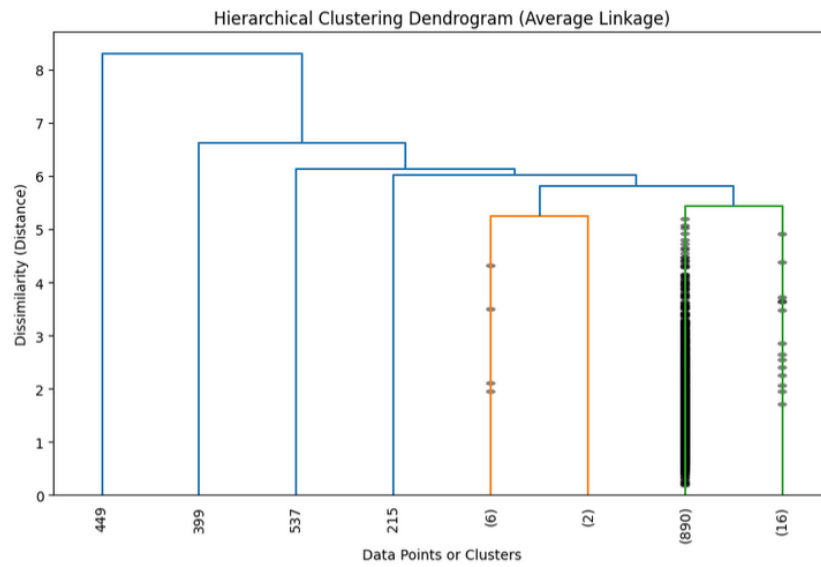
Single linkage



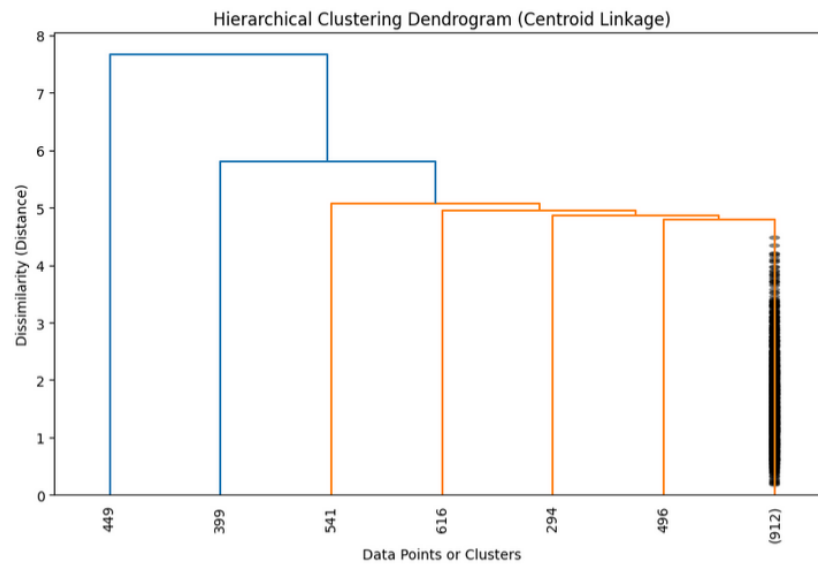
Complete Linkage



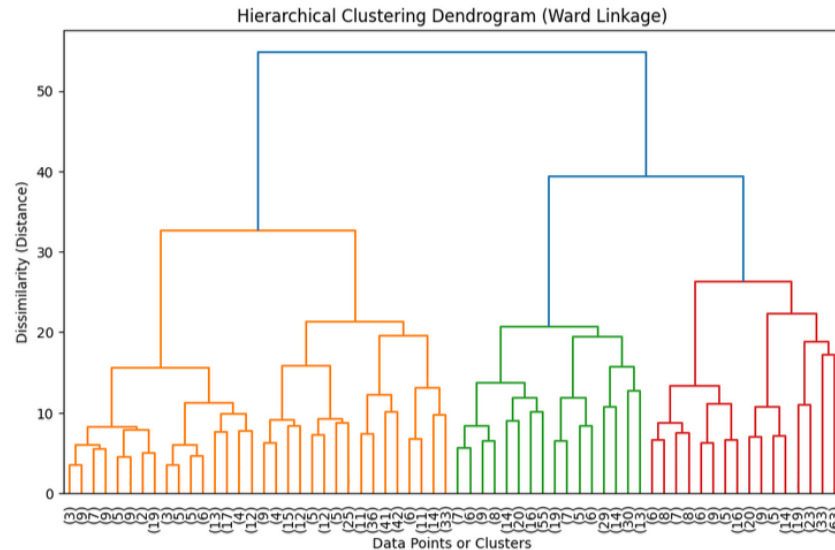
Average Linkage



Centroid Linkage



Ward Linkage



Decide linkage type

Now how to evaluate them according to which clustering linkage is better?

By Comparing the clustering with target splitting the hierarchy at level clusters = 2

Using a function `Compare_Clusters_with_target`:

```
# Compare clusters with target labels
# Add cluster labels and target labels to a DataFrame for comparison
def compare_clusters_with_target(cluster_labels):
    comparison_df = pd.DataFrame({
        'Cluster': cluster_labels,
        'Target': y
    })

    # Evaluate clustering performance
    # Count the number of data points in each cluster-target combination
    cluster_target_counts = comparison_df.groupby(['Cluster', 'Target']).size().unstack(fill_value=0)

    print("Cluster vs. Target Comparison:")
    print(cluster_target_counts)

    # Visualize the comparison
    cluster_target_counts.plot(kind='bar', stacked=True, figsize=(10, 6), colormap='viridis')
    plt.title('Cluster vs. Target Distribution')
    plt.xlabel('Cluster')
    plt.ylabel('Count')
    plt.legend(title='Target')
    plt.show()
```

The `compare_clusters_with_target` function compares clustering results with actual target labels. It creates a DataFrame showing the relationship between clusters and targets, displays a table of counts for each cluster-target combination, and generates a stacked bar chart for easy visualization. This helps evaluate how well

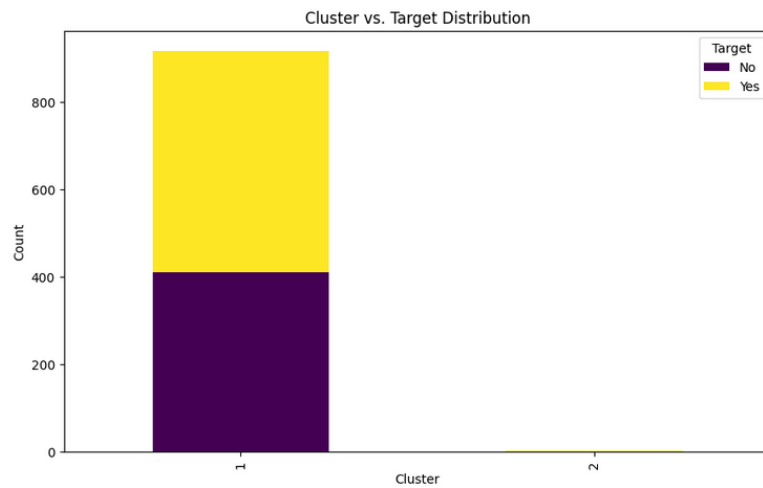
the clusters align with the true labels. and bar chart to visualize the distribution of true target labels within each cluster.

- **X-axis:** Represents clusters.
- **Y-axis:** Shows the count of data points in each cluster.
- **Stacked bars:** Represent the proportion of each target label in the corresponding cluster.

Single linkage

Cluster vs. Target Comparison:

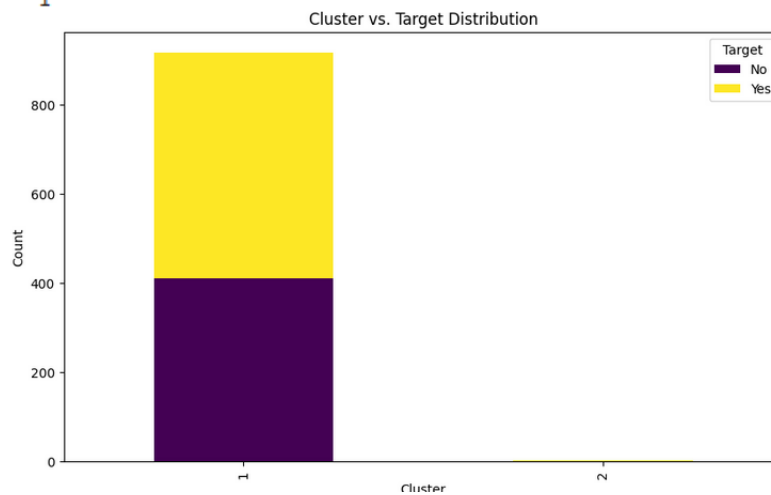
Target	No	Yes
Cluster		
1	410	507
2	0	1



Complete Linkage

Cluster vs. Target Comparison:

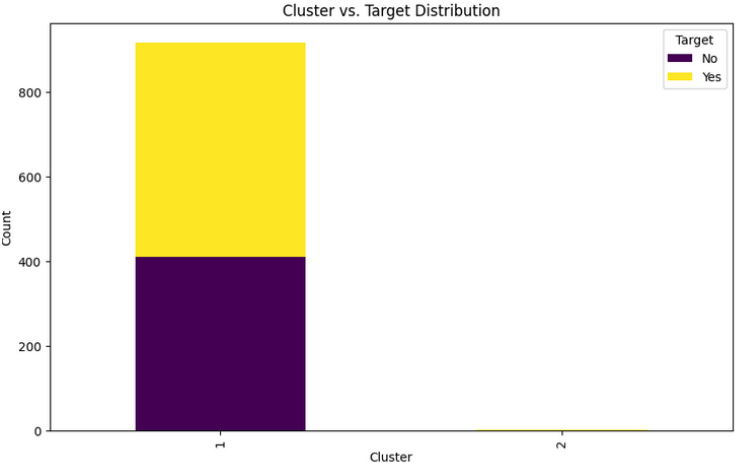
Target	No	Yes
Cluster		
1	410	507
2	0	1



Average Linkage

Cluster vs. Target Comparison:

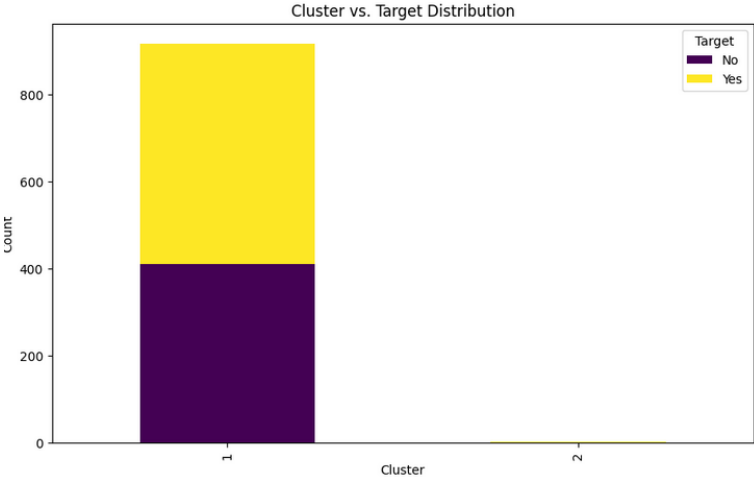
Target	No	Yes
Cluster		
1	410	507
2	0	1



Centroid Linkage

Cluster vs. Target Comparison:

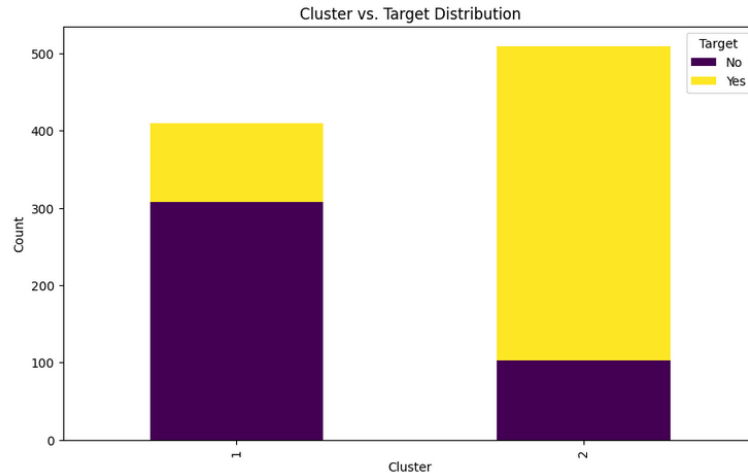
Target	No	Yes
Cluster		
1	410	507
2	0	1



Ward Linkage

Cluster vs. Target Comparison:

Target	No	Yes
Cluster		
1	307	102
2	103	406



Differences and similarities between Linkages

Single Linkage

- Shows a "chaining" effect where clusters are formed sequentially
- Has a relatively flat structure with most merges happening at similar heights
- Point 449 appears as an outlier, merging very late with other clusters

Complete Linkage

- Creates more balanced, evenly-sized clusters
- Shows clearer separation between different groups (visible through different colors)
- The hierarchical structure is more obvious and interpretable

Average Linkage

- Uses mean distance between clusters for merging
- Shows a more balanced structure than single linkage
- Y-axis scale ranges from 0-8
- Point 449 still appears as an outlier
- Shows clearer hierarchical structure than single/centroid linkage
- Has four main clusters visible from the branching pattern

Centroid Linkage

- Similar to single linkage but with smoother transitions

- Also shows point 449 as an outlier
- The clustering pattern is more gradual

Ward Linkage

- Creates the most visually distinct clusters
- Has the largest range on the y-axis (distance measure up to 50)
- Shows three main color-coded clusters (orange, green, and red)
- Produces more balanced cluster sizes

Y-axis Scale

Each method uses different distance scales:

- Single: ~0-6
- Complete: ~0-12
- Centroid: ~0-8

Use proximity measures is dissimilarity

For ward linkage dendrogram Analysis

Detailed Analysis of Ward's Method Dendrogram:

1. Cluster Structure:

- Shows three primary clusters (color-coded in orange, green, and red)
- The highest split occurs at distance ~55, separating the first major branch
- Second major split occurs around distance ~40

2. Cluster Characteristics:

- Left cluster (orange): Largest group, contains about 40% of data points
- Middle cluster (green): Medium-sized, more compact structure
- Right cluster (red): Shows internal sub-clustering with clear hierarchical divisions

3. Proximity Analysis:

- Intra-cluster distances are relatively small (below 20)
- Inter-cluster distances are large (above 30)
- The orange cluster shows more internal variation than the others
- The red cluster shows the most distinct sub-cluster structure

4. Notable Features:

- Clear separation between major clusters suggests well-defined groups
- Balanced cluster sizes indicate good clustering stability
- Height of merges provides clear indication of cluster distinctness
- Hierarchical structure is very well-defined with clear levels

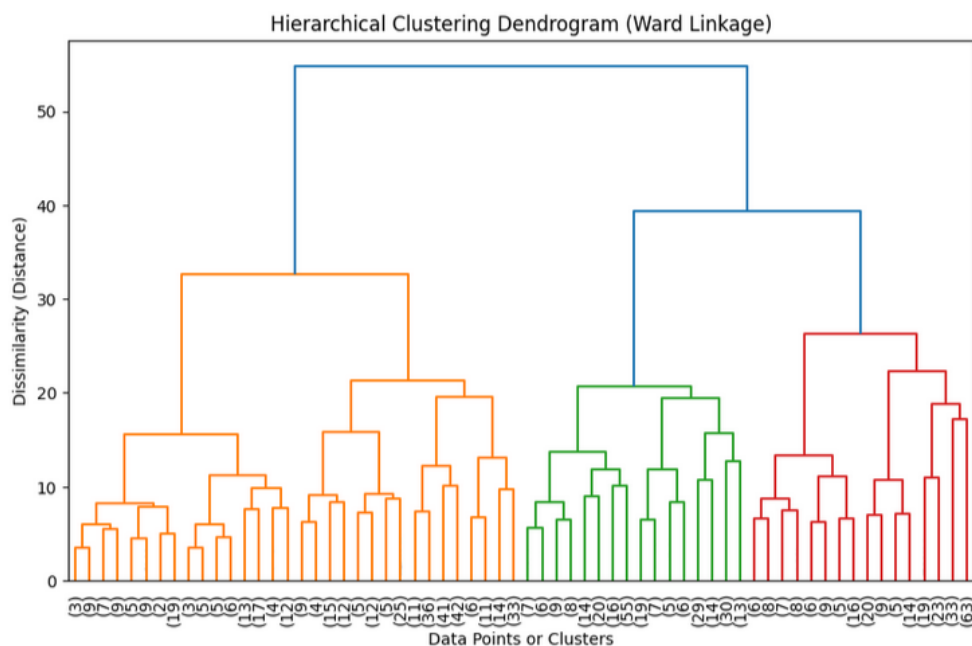
5. Optimal Cutting Point:

- Based on the structure, cutting at height ~ 30 would yield 3 main clusters
- Cutting at height ~ 20 would give about 5-6 sub-clusters
- The dendrogram suggests natural clustering at these levels due to large distance gaps

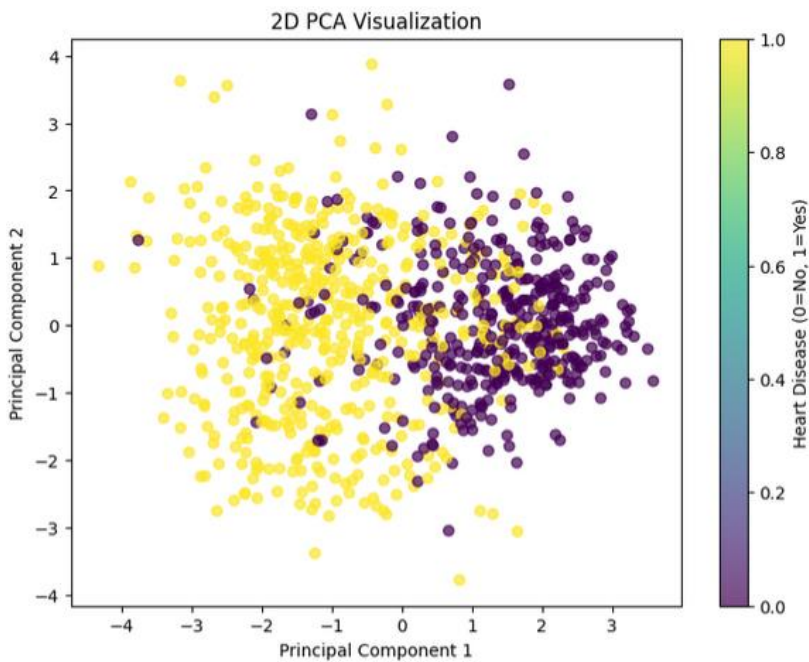
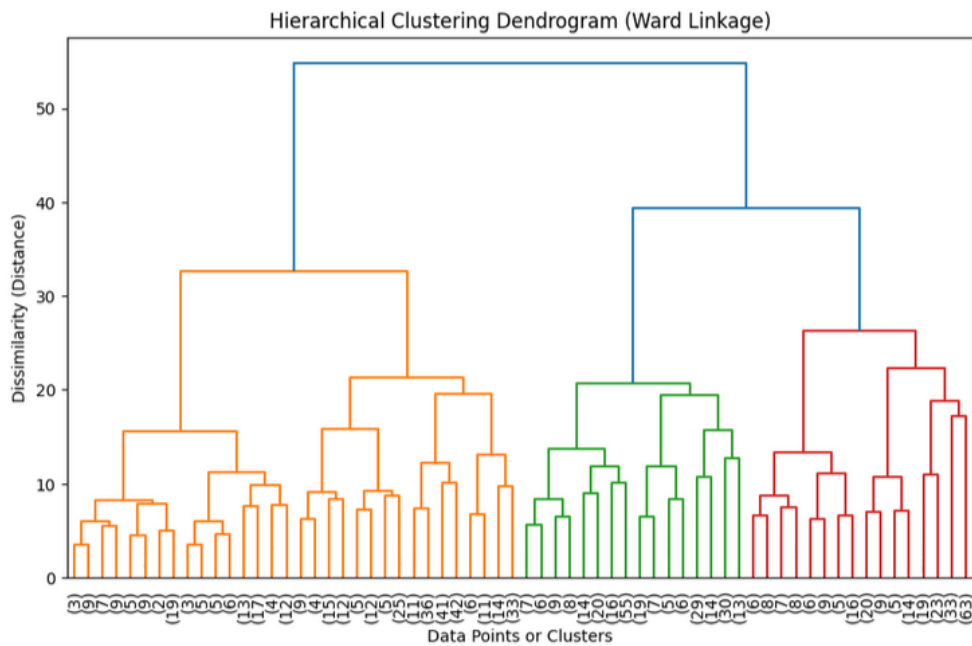
The Ward dendrogram appears to be the most informative for this dataset because:

- It shows clear, interpretable cluster separation
- The hierarchical structure is well-balanced
- It avoids the chaining effect seen in single linkage
- The height differences between merges are meaningful and interpretable
- It provides multiple levels of clustering that could be useful depending on the analysis needs

But in our case only we need 2 clusters so we'll cut between the ranges of 38 to 53



Hierarchical clustering insights with the PCA and classifier results



1. Structure Alignment:

- The dendrogram shows 3 distinct high-level clusters, while the PCA visualization primarily shows 2 main groups (purple/yellow)

- This apparent discrepancy might be explained by the fact that two of the dendrogram clusters (orange, green and red) could be more similar to each other in the principal component space, leading to their partial overlap in the PCA plot
- The hierarchical structure visible in the dendrogram (particularly in the orange cluster) is reflected in the PCA plot as varying densities and sub-groupings within the yellow points

2. Separation Quality:

- The height of the dendrogram's highest split (around 50 units) indicates significant dissimilarity between major clusters
- This strong separation is validated by the high classification accuracies (86-91%) across all models
- The SVM's superior performance (91.3%) suggests it's best at handling the particular boundary structure shown in both the dendrogram and PCA plot
- The overlap region in the PCA plot likely corresponds to the areas where classifiers make their mistakes, particularly visible in the center where purple and yellow points mix

3. Class Distribution:

- The dendrogram's asymmetric structure (larger orange cluster vs smaller green and red clusters) aligns with the class imbalance evident in the classifier support numbers (more "Yes" than "No" instances)
- This imbalance is also visible in the PCA plot, where one color group (yellow) appears to have more points than the other
- Despite this imbalance, all classifiers maintain good precision and recall, suggesting the clusters are well-defined despite size differences

4. Boundary Complexity:

- The hierarchical nature of the dendrogram clusters suggests complex decision boundaries
- This is reflected in the PCA plot's non-linear separation between classes
- The strong performance of SVM makes sense given its ability to handle such complex boundaries through kernel functions

- The slightly lower performance of Naive Bayes (86.4%) might be explained by its assumption of simple decision boundaries, which doesn't align with the complex structure shown in both visualizations

The hierarchical clustering reveals the nested structure of the data, the PCA shows the global distribution and separation of classes, and the classifier performances validate that these patterns are meaningful and predictable. The consistency across these different views strengthens confidence in the underlying data structure and the reliability of the classification results.

The hierarchical nature of clusters in the dendrogram suggests varying densities of points, which KNN can handle well

Its slightly lower performance compared to SVM might be due to the mixing zones in the PCA plot where neighborhoods overlap

Decision Tree - 87.5% Accuracy:

The hierarchical splitting nature of decision trees aligns well with the dendrogram's tree structure

9. Appendices

[1] **ChestPainType** values representing different types of chest pain, as follows:

ATA (Atypical Angina): This refers to chest pain that is not typical of angina. It can be caused by a variety of factors and might not follow the usual pattern of angina pain.

NAP (Non-Anginal Pain): This type of chest pain is not related to heart disease. It might arise from other conditions like digestive issues or musculoskeletal problems.

ASY (Asymptomatic): This refers to a lack of symptoms. Individuals with asymptomatic chest pain may not feel any discomfort, even if they have underlying heart disease.

TA (Typical Angina): This refers to chest pain caused by restricted blood flow to the heart, often triggered by physical exertion or stress. It is a common symptom of coronary artery disease.

[2] **RestingECG** column values:

Normal: The electrocardiogram shows normal heart rhythm with no signs of ischemia or abnormal electrical activity.

ST: This indicates ST-segment depression or elevation, which is often associated with myocardial ischemia (reduced blood flow to the heart).

LVH (Left Ventricular Hypertrophy): refers to thickening of the heart's left ventricle, which can occur because of high blood pressure or other heart conditions.